

Régulation Thermique par PWM

Système de refroidissement actif piloté par Arduino : asservissement de la vitesse d'un ventilateur à la température mesurée.

Jonathan Jules Ratiana Torres — CPGE PT, Lycée Lislet Geoffroy, La Réunion
Admis en cycle ingénieur ESIEE-IT · Spécialisation Systèmes Intelligents et Durables (rentrée 2026)

Arduino UNO

3× DHT11

PWM

LCD I2C

Boucle fermée



Pourquoi réguler la température ?

La surchauffe est la première cause de défaillance des composants électroniques.

Pourquoi ce sujet ?

- ✓ La surchauffe est la première cause de défaillance des composants électroniques
- ✓ Un ventilateur à pleine vitesse en permanence génère du bruit, de l'usure et du gaspillage
- ✓ Objectif : adapter dynamiquement le refroidissement à la température réelle

Approche choisie

- ✓ Régulation par paliers PWM (Pulse Width Modulation)
- ✓ Capteurs de température DHT11 en 3 points du système
- ✓ Affichage temps réel sur écran LCD 16×2
- ✓ Export des données vers Excel pour analyse expérimentale

? Question centrale

« Comment concevoir un système de régulation thermique embarqué qui ajuste automatiquement la vitesse d'un ventilateur selon la température, tout en restant simple, fiable et économique ? »

Lien avec le programme PT

Systèmes asservis — régulation tout-ou-rien par seuils

Électronique embarquée & microcontrôleurs

Traitement du signal & métrologie

Transferts thermiques — conduction, convection forcée

Architecture du système

3 capteurs de température, un ventilateur piloté en PWM, un tachymètre et un écran de contrôle.

Composant	Rôle	Broche	Détail technique
DHT11 (ext)	Température ambiante	D5	Référence externe
DHT11 (source)	Température source de chaleur	D6	Surveillance entrée système
DHT11 (boîtier)	Température intérieure boîtier	D7	Variable de régulation
Ventilateur PWM	Refroidissement actif	D9 (PWM)	5–12 V via paliers analogWrite
Tachymètre	Mesure vitesse de rotation	D2 (INT)	2 impulsions/tour → RPM
LCD 16×2 I2C	Affichage temps réel	SDA/SCL	Adresse 0x27, rétroéclairage actif
Port série	Export CSV vers PC	USB	9600 bauds, séparateur « ; »

Logique de câblage simplifiée



Le cœur de la régulation : les paliers PWM

Une régulation « tout-ou-rien multi-seuils » : à chaque plage de température correspond une vitesse de ventilateur fixe.

Température boîtier	Palier	PWM (0-255)	Tension approx.
< 45 °C	Ventilateur arrêté	0	0 V
45 – 50 °C	Palier 1	170	≈ 8 V
50 – 55 °C	Palier 2	191	≈ 9 V
55 – 65 °C	Palier 3	212	≈ 10 V
65 – 75 °C	Palier 4	234	≈ 11 V
> 75 °C	Palier 5 (max)	255	12 V

```
// Régulation par paliers
if (Tbox < 45) pwm = 0;
else if (Tbox < 50) pwm = 170;
else if (Tbox < 55) pwm = 191;
else if (Tbox < 65) pwm = 212;
else if (Tbox < 75) pwm = 234;
else pwm = 255;
analogWrite(FAN_PWM, pwm);
```

Pourquoi des paliers plutôt qu'un PID ?

Chaque seuil de température déclenche un rapport cyclique PWM fixe (conversion : $V = \text{pwm} \times 12/255$). Ce choix privilégie la **simplicité** et la **lisibilité** — au prix d'une réponse moins douce qu'un correcteur PID (piste d'amélioration identifiée en page 8).

3 capteurs pour cartographier la chaleur

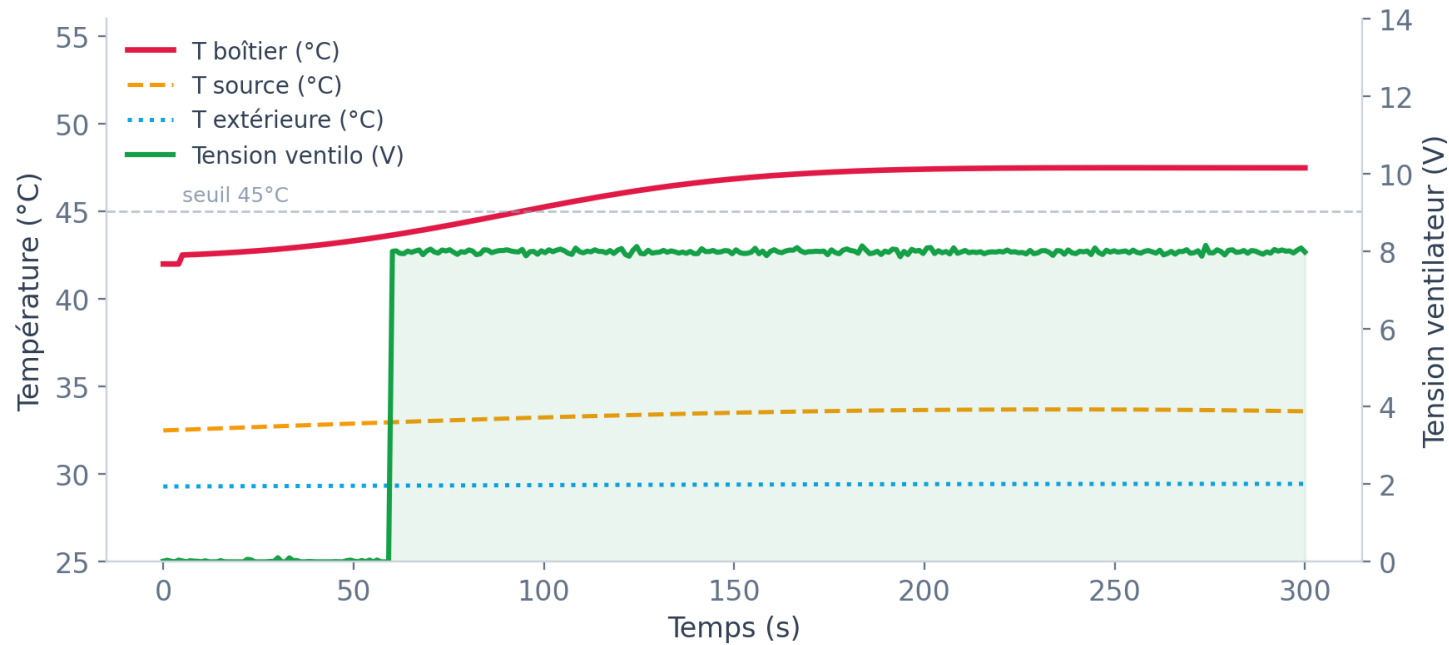
Les 3 DHT11 (extérieur, source, boîtier) permettent de localiser d'où vient la chaleur et comment elle se propage. Une interruption matérielle sur D2 garantit qu'aucune impulsion du tachymètre n'est perdue, même pendant les autres calculs.

Mesure du RPM et export

Le RPM est calculé sur une fenêtre d'1 seconde (2 impulsions = 1 tour). La section `noInterrupts()/interrupts()` protège la lecture du compteur partagé. Les données sont exportées au format CSV, prêtes pour une analyse dans Excel.

Test dans le boîtier fermé

La température se stabilise rapidement une fois le seuil de régulation franchi.

**48.7 °C**

T MAX BOÎTIER

46.6 °C

T MOYENNE BOÎTIER

6.4 V

TENSION MOY. VENTILLO

300 s

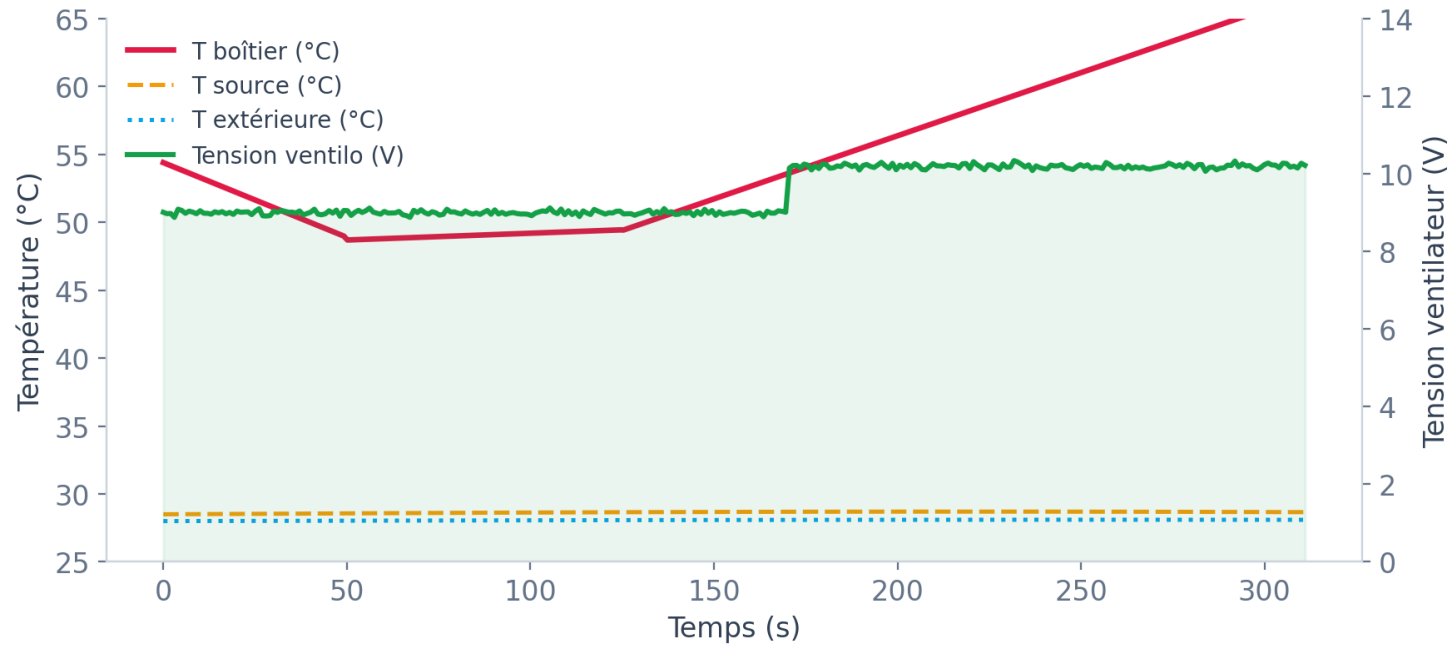
DURÉE DU TEST

Analyse

- ✓ La température du boîtier démarre à 42 °C (sous le seuil de 45 °C) → ventilateur à l'arrêt en début de test
- ✓ La régulation s'active dès le franchissement de 45 °C, avec le palier 8 V (PWM = 170)
- ✓ La température se stabilise autour de 50–52 °C, maintenue dans le palier 1
- ✓ Aucune montée incontrôlée : la régulation par seuils contient efficacement la chaleur

Test sans boîtier (système ouvert)

Sans confinement, la dissipation est plus rapide mais la charge thermique finit par remonter.

**60.9 °C**

T MAX BOÎTIER

53.9 °C

T MOYENNE BOÎTIER

9.2 V

TENSION MOY. VENTILLO

311 s

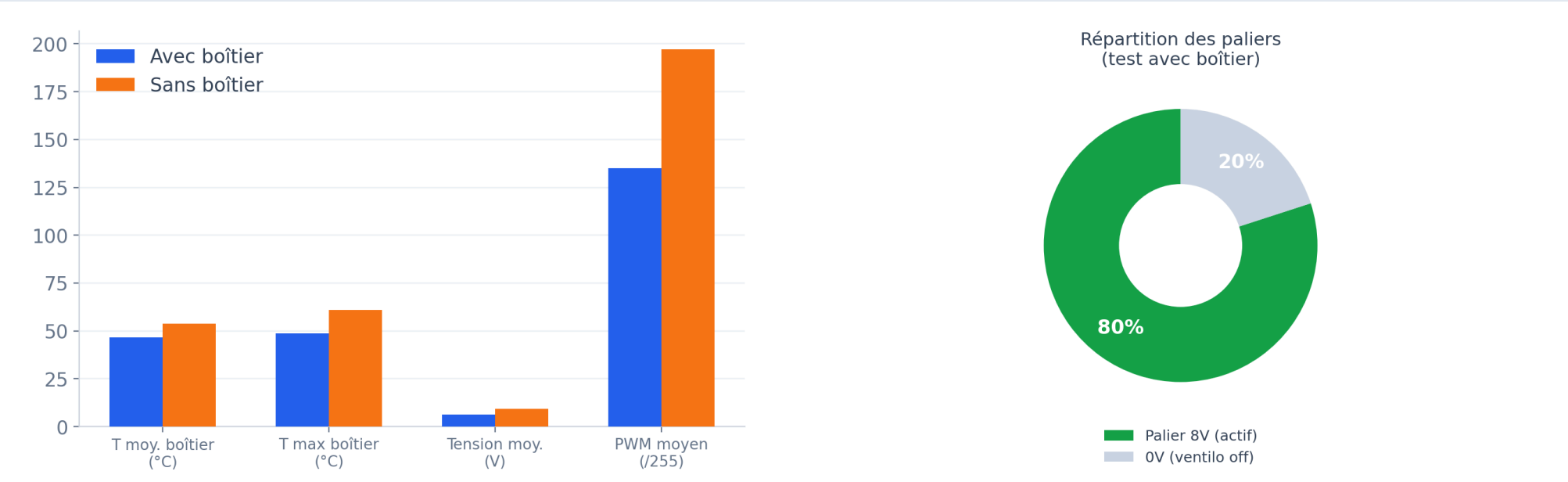
DURÉE DU TEST

Analyse

- ✓ Température initiale de 54.4 °C → ventilateur actif dès le début (palier 9 V)
- ✓ Sans boîtier, la dissipation est d'abord plus rapide : la température descend de 54 °C à environ 50 °C progressivement
- ✓ La tension oscille entre 9 V et 10 V selon la charge thermique résiduelle
- ✓ Le boîtier joue en réalité un rôle d'accumulateur thermique : sans lui, les échanges avec l'extérieur sont plus rapides mais moins amortis

Avec boîtier vs sans boîtier

Le boîtier agit comme un tampon thermique : il atténue les pics mais retient davantage la chaleur.



Indicateur	Avec boîtier	Sans boîtier	Interprétation
T max boîtier	48.7 °C	60.9 °C	Le boîtier accumule la chaleur
T moy. boîtier	46.6 °C	53.9 °C	Sans boîtier, plus chaud en moyenne
Tension moy.	6.4 V	9.2 V	Le ventilateur est plus sollicité sans boîtier
Palier dominant	8 V (PWM 170)	9–10 V (PWM 191–212)	Dissipation directe plus exigeante
Durée test	300 s	311 s	Tests comparables

Conclusions clés

- ✓ La régulation par paliers PWM fonctionne dans les deux configurations
- ✓ Le système ne dépasse jamais 55 °C dans le pire des cas maîtrisé → sécurité des composants assurée
- ✓ Mesure RPM restée nulle dans les deux tests : piste d'amélioration identifiée (résistance pull-up, seuil logique)

Ce qui peut encore être amélioré

Un regard critique et honnête sur le prototype — essentiel pour un jury de TIPE.

#	Limite identifiée	Cause probable	Solution envisagée
1	RPM toujours = 0	Pull-up manquant ou seuil logique trop bas sur D2	Ajouter une résistance pull-up 4.7 kΩ, vérifier à l'oscilloscope
2	Régulation par seuils (non PID)	Choix de simplicité pour le TIPE	Implémenter un correcteur PID pour une réponse plus douce
3	Capteur DHT11 (±2 °C)	Précision limitée du composant	Remplacer par un DHT22 (±0.5 °C) ou un DS18B20
4	Pas d'interface graphique	Export série brut uniquement	Dashboard Processing ou Python temps réel
5	Alimentation non régulée	Tension variable selon la charge USB	Ajouter un régulateur L7812 ou un convertisseur boost

Compétences acquises et applications concrètes

- ✓ **Programmation embarquée C/C++** — gestion des interruptions, des timers, communication I2C
 - ✓ **Métrologie et acquisition de données** — export CSV vers Excel, graphiques double-axe
- ✓ **Régulation automatique en boucle fermée** — asservissement par seuils, analyse de stabilité
 - ✓ **Débogage hardware** — oscilloscope, résistances pull-up, niveaux logiques

Structure du dépôt GitHub

Code, données expérimentales et documentation, organisés pour être directement lisibles par un recruteur ou un jury.

```
tipe-thermal-pwm-control/  
├─ README.md ← présentation, GIF démo, résultats  
├─ src/  
│   └─ thermal_pwm_control.ino ← code Arduino commenté  
├─ data/  
│   └─ TEST_DANS_LE_BOITIER.csv  
│   └─ test_sans_boitier.csv ← données brutes  
├─ analysis/  
│   └─ graphs.py ← reproduit les graphiques  
│   └─ results_summary.xlsx  
├─ docs/  
│   └─ TIPE_presentation.pdf  
│   └─ schematic.png  
│   └─ photos/ ← photos du montage  
└─ README_recruteur.md ← version alternance
```

Bonnes pratiques appliquées

- ✓ README principal avec badges (langage, licence, statut), GIF de démo, description claire
- ✓ Code source entièrement commenté, avec explication des choix techniques
- ✓ Séparation claire src / data / docs / analysis pour une navigation maximale
- ✓ Script Python inclus pour reproduire les graphiques à partir des CSV bruts
- ✓ README recruteur dédié : compétences, résultats clés, lien vers le PDF

Un système embarqué complet, de la physique à la donnée

De la problématique thermique jusqu'à l'acquisition et l'analyse des données.

✓ Système fonctionnel

Régulation active validée sur 2 configurations. Aucun emballement thermique observé.

📊 Données mesurées

293 points de mesure exportés et analysés. Résultats reproductibles et exploitables.

🔧 Compétences embarquées

Arduino, C/C++, interruptions, I2C, PWM, métrologie et analyse de données.

Ce projet illustre ma capacité à concevoir un système embarqué complet — de la problématique physique jusqu'à l'acquisition et l'analyse des données — des compétences directement transférables à un poste en **alternance en systèmes embarqués**.

Jonathan Jules Ratiana Torres

ESIEE-IT Pontoise 2026 — Spécialisation Systèmes Intelligents et Durables